

Ottimizzazione dei modelli di deep learning

andrea.simonelli

April 2026

1 Introduction

Per questo problema di classificazione multilabel si è scelto di utilizzare tre modelli di rete neurale. Queste reti devono essere in grado di prendere in ingresso testi precedentemente tokenizzati attraverso un layer TextVectorization di keras. Questo layer riceve i testi e crea un vocabolario di interi a partire dalle singole parole. Si deve poi impostare la lunghezza dell'output; questo viene fatto osservando la distribuzione delle lunghezze dei testi(fig:1). Nel caso in cui il testo di un articolo sia più corto, viene eseguito il padding; viceversa il testo viene troncato. I testi sono prima di tutto ripuliti da eventuali caratteri che possono mandare in confusione la rete e dalle stopwords(congiunzioni, articoli, etc).

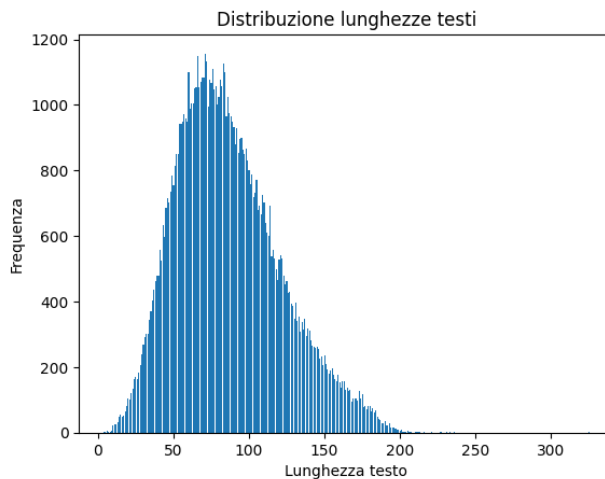


Figure 1: Distribuzione dalla lunghezza dei testi tokenizzati.

Il dataset (testi e labels) viene poi diviso in training, validation e test (80%, 10%, 10%) con la funzione di sklearn train_test_split(). Tutti i modelli presentano un layer Embedding iniziale, che esegue l'embedding dei testi che saranno processati dai layer successivi. Viene utilizzata come funzione di loss la BinaryCrossentropy, come optimizer adam e vengono valutate le metriche di Precision, Recall e F1-Score. In keras sono disponibili due average della metrica F1-Score, per tutti i modelli viene utilizzato l'average micro. Durante il training dei modelli la soglia delle metriche è impostata a 0.5, per le predizioni si valutano diversi valori di soglia.

La Precision è definita come:

$$Precision = \frac{TruePositives}{TruePositives + FalsePositives} \quad (1)$$

La Recall è calcolata come:

$$Recall = \frac{TruePositives}{TruePositives + FalseNegatives} \quad (2)$$

Infine l’F1-Score è dato da:

$$F1 - Score = \frac{2 \cdot Precision \cdot Recall}{Precision + Recall} \quad (3)$$

La Precision quindi indica quante sono le label corrette che la rete riesce ad identificare, tra tutte quelle che sono state predette. La Recall dice invece quanto la rete è prudente, ovvero dice quanto la rete preferisce non assegnare una categoria all’esempio. L’F1-Score è ottenuto attraverso la media armonica tra i due, indica infatti il bilanciamento tra le due metriche.

Tutti i modelli hanno come layer finale un layer di tipo Dense di dimensione pari alla dimensione dei labels(567) e come funzione di attivazione la sigmoid. Questa funzione restituirà un valore in un intervallo $[0, 1]$ che indica la probabilità che l’esempio fornito alla rete appartenga o meno a quella categoria. Di conseguenza saranno necessari 567 neuroni, uno per ogni categoria possibile.

2 Modello Dense

Il primo modello è chiamato Dense, in quanto presenta principalmente layer di tipo Dense. Il modello, implementato in modalità Sequential, è composto da:

- Embedding(input_dim=10001, output_dim=512)
- GlobalAveragePooling1D()
- Dense(512, activation='relu')
- Dense(567, activation='sigmoid')

Come già detto la loss utilizzata è la BinaryCrossentropy, come optimizer si utilizza adam e come metriche si osservano Precision, Recall e F1-Score. Per il training si utilizza un batch size di 256 e si impostano 50 epoche.

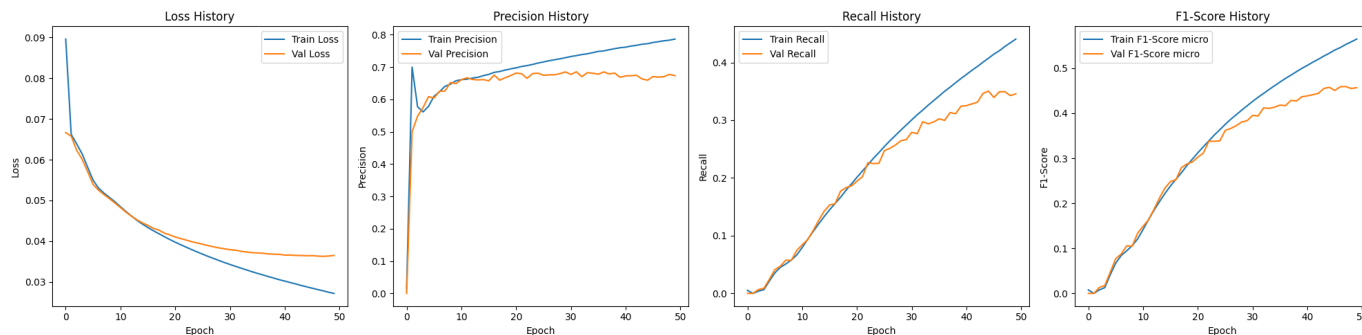


Figure 2: Curve di validazione e training di Loss, Precision, Recall e F1-Score(da sinistra a destra).

Come si può vedere dai grafici di loss, Precision, Recall e F1-Score (fig:2) allenando la rete per 50 epoche si osserva overfitting intorno alla ventesima epoca. Si implementano quindi i callbacks di EarlyStopping e ReduceLROnPlateau. Il primo, impostato con una patience di 10, reimposta i pesi del modello all’epoca del valore migliore di loss ottenuto. Il ReduceLROnPlateau viene impostato con una patience di 4 e un fattore di riduzione del learning rate di 0.2. Viene scelta una patience di 4 per far sì che il callback possa agire almeno un paio di volte prima che l’EarlyStopping intervenga. Inoltre, si aumenta la dimensione del penultimo layer Dense da 512 a 1024. L’ultimo layer Dense avrà una dimensione pari alla dimensione dei label (567), aumentando la dimensione del layer precedente si evitano possibili bottleneck.

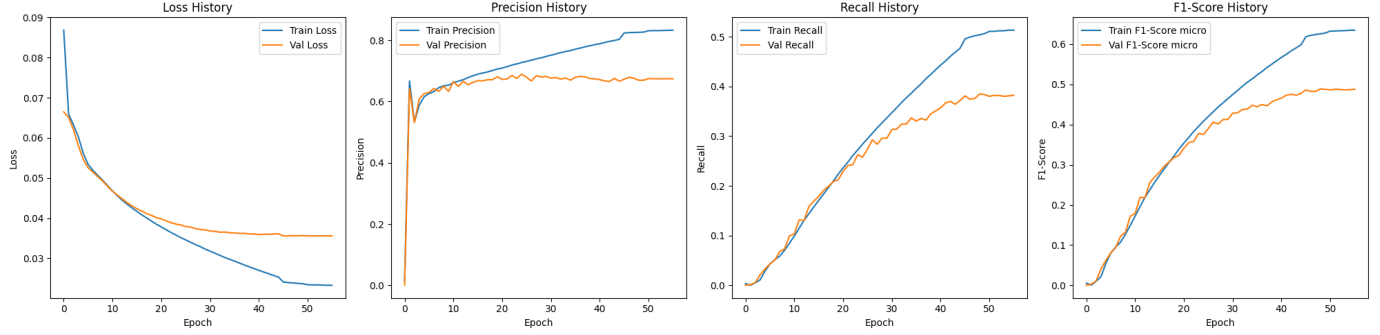


Figure 3: Curve di validazione e training di Loss, Precision, Recall e F1-Score(da sinistra a destra).

Come atteso l'EarlyStopping ha fermato l'allenamento della rete prima del massimo di epoche impostate e sono stati ripristinati i pesi dell'epoca 46. Sui dati di validazione si ottengono i seguenti valori di loss e delle metriche:

Loss: 0.0356, Precision: 0.6723, Recall: 0.3809 e F1-Score: 0.4863.

Per osservare se il modello ha una buona generalizzazione si valuta il modello sui dati di test, e si ottiene:

Loss: 0.0359, Precision: 0.6708, Recall: 0.3776 e F1-Score: 0.4832.

A questo punto si sostituisce il layer GlobalAveragePooling1D con il GlobalMaxPooling1D. Il GlobalAveragePooling1D esegue una media dei valori che riceve in input, di conseguenza un segnale forte che può aiutare la rete a imparare il pattern giusto, viene mitigato. In particolare, come in questo caso, il layer TextVectorization esegue il padding dei testi più corti della dimensione di output impostata. Di conseguenza dopo l'embedding molti testi avranno vettori con molti zeri, che attenuano i segnali importanti per l'apprendimento quando sono processati dal GlobalAveragePooling1D. Al contrario il layer GlobalMaxPooling1D seleziona solamente il segnale più forte, agendo come una sorta di filtro per gli zeri di padding.

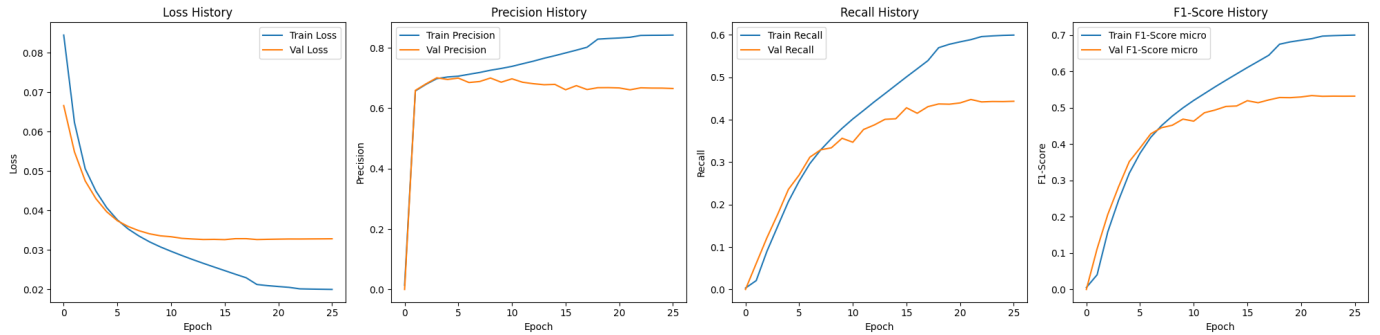


Figure 4: Curve di validazione e training di Loss, Precision, Recall e F1-Score(da sinistra a destra).

Con questa modifica sui dati di validazione si ottengono i seguenti valori di loss e delle metriche, relative all'epoca 16:

Loss: 0.0326, Precision; 0.6611, Recall: 0,4281 e F1-Score: 0.5197

Invece, sui dati di test si ottiene:

Loss: 0.0330, Precision; 0.6632, Recall: 0,4265 e F1-Score: 0.5191

Rispetto alla configurazione precedente si ha un aumento di Recall, mentre la Precision rimane pressochè invariata. Questo porta a valori di F1-Score più alti.

Successivamente si aumenta la capacità della rete inserendo un ulteriore layer Dense uguale al primo e aumentando il vocabolario del TextVectorization, di conseguenza si aumenta anche la dimensione di input del layer Embedding a 25001. Per stabilizzare gli output dei layer vengono inseriti dei BatchNormalization

dopo i layer Dense e GlobalMaxPooling1D. Questo viene fatto per contrastare in fenomeno dell'Internal Covariate Shift e per ridurre il Vanishing Gradient che può verificarsi con l'aumento della profondità della rete. Il layer BatchNormalization calcola la media e la deviazione standard del batch. Gli output del layer precedente vengono normalizzati sottraendo la media calcolata e dividendo per la deviazione standard. Per far sì che il modello riporti una buona generalizzazione si implementa un layer di Dropout tra i due hidden layers Dense. Questo layer spegne casualmente i neuroni dei layer Dense in modo tale che la rete fatichi ad apprendere pattern specifici per i dati di training, aumentando la generalizzazione. Il modello avrà quindi una struttura composta da:

- Embedding(input_dim=25001, output_dim=512)
- GlobalAveragePooling1D()
- BatchNormalization()
- Dense(1024, activation='relu')
- BatchNormalization()
- Dropout(0.3)
- Dense(1024, activation='relu')
- BatchNormalization()
- Dense(567, activation='sigmoid')

Allenando il modello così modificato, l'EarlyStopping ripristina i pesi dell'epoca 14. A questo punto dell'allenamento si ottengono i valori di validazione di:

Loss: 0.0327, Precision: 0.6724, Recall: 0.4739 e F1-Score: 0.5560

Sui dati di test si ricava:

Loss: 0.0330, Precision: 0.6743, Recall: 0.4752 e F1-Score: 0.5575

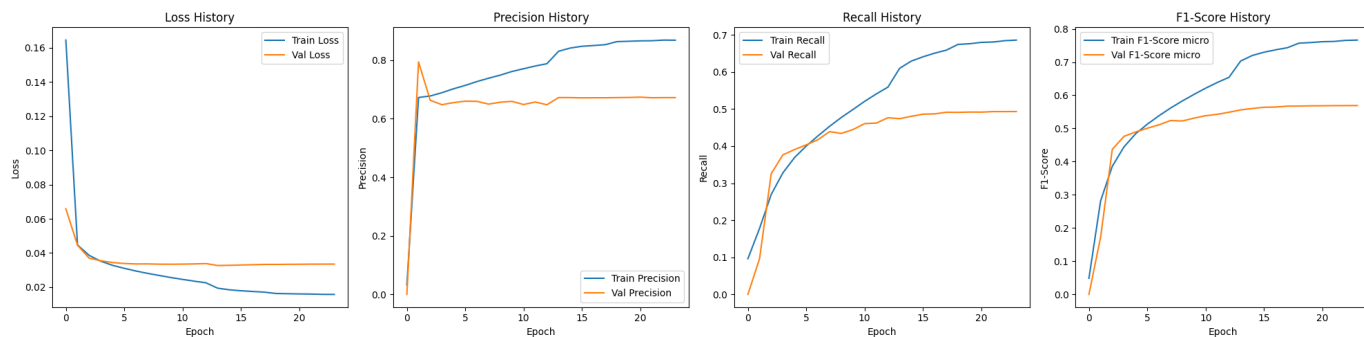


Figure 5: Curve di validazione e training di Loss, Precision, Recall e F1-Score(da sinistra a destra).

Con questa configurazione si osserva un ulteriore aumento di Recall. Si osserva anche un leggero aumento di Precision che però è poco significativo, in quanto può essere dovuto alla stocasticità del processo di training della rete.

Avendo un totale di 567 possibili label, se la rete predice molti zeri la Loss sarà comunque bassa. Per problemi di multi-label, è consigliato l'uso della Binary Focal Crossentropy come funzione di Loss. Questo tipo di Loss function introduce due parametri γ e α che penalizzano maggiormente l'apprendimento della rete se commette degli errori, costringendola a modificare i pesi per correggerli. Vengono valutati valori diversi di γ e α e si osservano i valori delle metriche ottenuti.

γ	α	Loss	Precision	Recall	F1-Score
2.0	0.25	0.0095	0.6732	0.4302	0.5250
3.0	0.45	0.0053	0.6691	0.4070	0.5062
4.0	0.65	0.0027	0.6540	0.3943	0.4920
5.0	0.90	0.0016	0.6587	0.2985	0.4108

Table 1: Valori di Loss, Precision, Recall e F1-Score al variare dei parametri γ e α .

Utilizzando la Binary Focal Crossentropy si osserva che aumentando i valori dei parametri si ottengono valori delle metriche più bassi. In particolare, viene penalizzata maggiormente la Recall. Si decide quindi di tenere il parametro γ a 2.0 e si inseriscono diversi valori di α . Il parametro α dà più importanza ad errori commessi nel non predire una categoria esistente, se viene impostato a valori maggiori di 0.5.

α	Loss	Precision	Recall	F1-Score
0.25	0.0095	0.6732	0.4302	0.5250
0.50	0.0095	0.6756	0.4411	0.5337
0.75	0.0096	0.6769	0.4332	0.5283
0.90	0.0095	0.6696	0.4437	0.5338

Table 2: Valori di Loss, Precision, Recall e F1-Score al variare del parametro α fissato γ .

Anche fissando γ e aumentando il parametro α , che penalizza maggiormente la rete quando sbaglia un positivo, non si riscontrano valori di F1-Score migliori rispetto ad utilizzare la BinaryCrossentropy. Quindi si decide di tornare ad utilizzare la BinaryCrossentropy come funzione di Loss e si valuta sui dati di test il modello allenato, variando la soglia decisionale delle metriche (tab:4, fig:6). Per ogni soglia si valuta anche il numero medio di keywords che la rete predice da confrontare con quelle del dataset originale pari a 7.73 (tab:3).

Soglia	0.30	0.35	0.40	0.45	0.50	0.55	0.60
Media	7.92	7.16	6.52	5.96	5.45	4.99	4.55

Table 3: Valori della media di keywords assegnate agli articoli dei dati di test.

Soglia	Precision	Recall	F1-Score
0.30	0.5714	0.5856	0.5784
0.35	0.5995	0.5553	0.5765
0.40	0.6269	0.5285	0.5735
0.45	0.6510	0.5018	0.5667
0.50	0.6743	0.4752	0.5575
0.55	0.6955	0.4491	0.5458
0.60	0.7152	0.4214	0.5303

Table 4: Tabella dei valori delle metriche al variare della soglia decisionale.

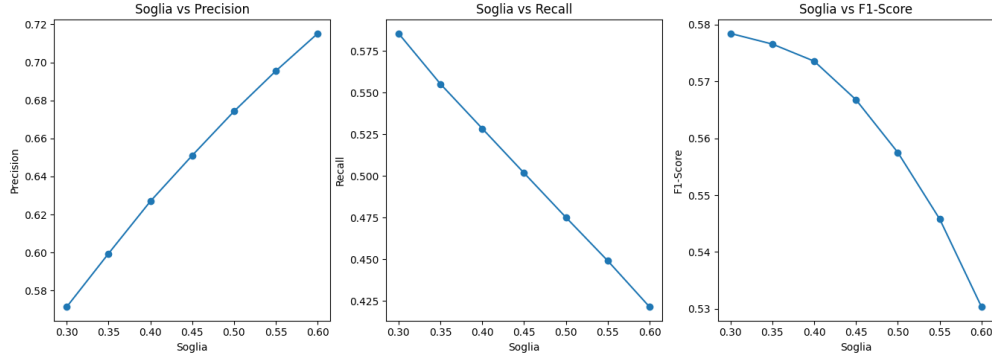


Figure 6: Grafici dei valori delle metriche Precision, Recall e F1-Score al variare della soglia decisionale.

Dalle tabelle e dai grafici si osserva che il valore di soglia ottimale per il modello è 0.3. Questo perchè, non solo per quel valore si ha una media simile di keywords assegnate agli articoli del dataset di test, ma l'F1-Score risulta maggiore. Infatti, anche se per una soglia di 0.3 si ha una diminuzione di Precision, l'aumento di Recall compensa questo calo. Di seguito vengono riportate alcune delle predizioni, sia alla soglia impostata durante l'allenamento (0.5) sia a quella ottimale (0.3).

```
Keywords predette con soglia 0.5: ('amplitude analysis', 'ckm matrix')
Keywords predette con soglia 0.3: ('amplitude analysis', 'b > k pi', 'background', 'ckm matrix', 'meson decay', 'numerical calculations', 'unitarity')
Keywords originali : ['amplitude analysis', 'b > k pi', 'sensitivity', 'numerical calculations', 'ckm matrix']
```

Figure 7: Predizione del modello Dense alla soglia di allenamento(0.5) e ottimale(0.3).

```
Keywords predette con soglia 0.5: ('higher order 1', 'quantum chromodynamics perturbation theory', 'supersymmetry')
Keywords predette con soglia 0.3: ('collinear', 'gauge field theory', 'higher order 1', 'quantum chromodynamics perturbation theory', 'supersymmetry')
Keywords originali : ['quantum chromodynamics perturbation theory', 'scattering', 'collinear', 'singlet', 'tensor energy momentum', 'infrared']
```

Figure 8: Predizione del modello Dense alla soglia di allenamento(0.5) e ottimale(0.3).

```
Keywords predette con soglia 0.5: ('cern lhc coll', 'parton showers', 'quantum chromodynamics')
Keywords predette con soglia 0.3: ('cern lhc coll', 'jet production', 'p p scattering', 'parton showers', 'quantum chromodynamics', 'rapidity')
Keywords originali : ['cern lhc coll', 'quantum chromodynamics', 'jet production', 'parton showers', 'gluon']
```

Figure 9: Predizione del modello Dense alla soglia di allenamento(0.5) e ottimale(0.3).

3 Modello CNN

Il secondo modello è composto da una parte convoluzionale, formata da layer di tipo Conv1D, e un classificatore formato da layer di tipo Dense. Tra queste due parti è inserito un layer di GlobalMaxPooling1D e come primo layer della struttura è utilizzato un layer Embedding. Nel dettaglio, il modello implementato in modalità Sequential è composto da:

- Embedding(input_dim=10001, output_dim=512)
- Conv1D(filters=128, kernel_size=5, activation='relu', padding='same')
- GlobalMaxPooling1D()
- Dense(512, activation='relu')
- Dense(567, activation='sigmoid')

Anche in questo caso si utilizza come funzione di loss la BinaryCrossentropy, come optimizer adam e si valutano le metriche di Precision, Recall e F1-Score con average micro. Inizialmente sono state impostate 30 epoche di training e un batch size di 256.

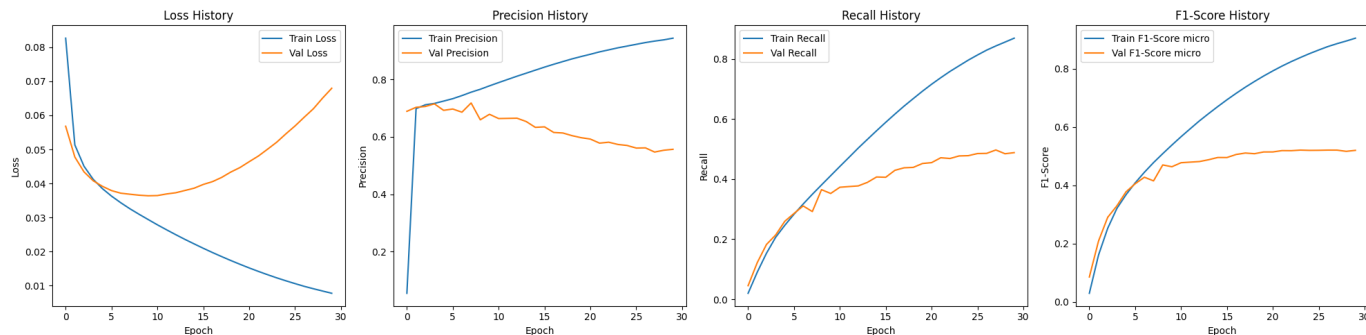


Figure 10: Curve di validazione e training di Loss, Precision, Recall e F1-Score(da sinistra a destra).

Dai grafici in figura 10, si osserva overfitting già intorno alla quinta epoca. Per contrastare l'overfitting si aggiungono layer di BatchNormalization dopo i layer convoluzionali, Dense e di pooling. Inoltre, tra i layer Dense viene inserito un layer di Dropout per aumentare la generalizzazione della rete, evitando che impari relazioni specifiche solo per i dati di training.

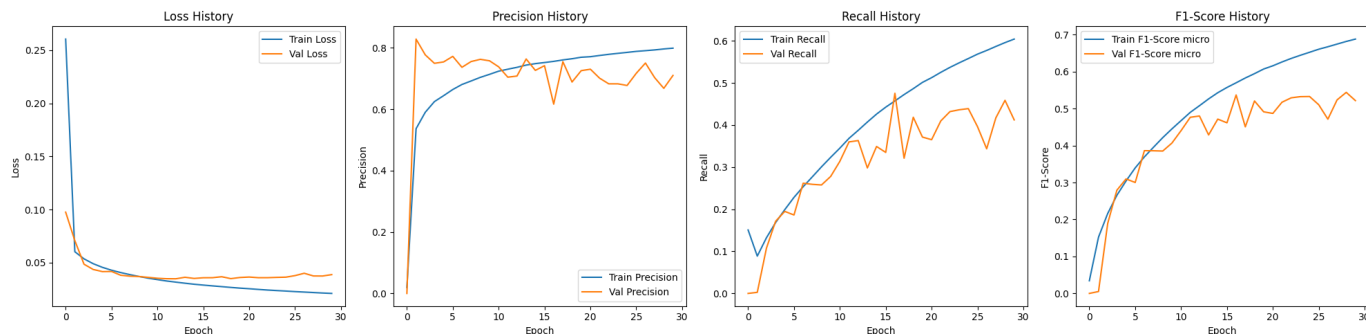


Figure 11: Curve di validazione e training di Loss, Precision, Recall e F1-Score(da sinistra a destra).

Dalle curve di validazione e training(fig:11) si osserva l'efficacia dei layer inseriti. Nelle curve di loss si nota come il gap si sia ridotto, così come nelle curve delle metriche. In quest'ultime si osservano però delle forti oscillazioni che potrebbero essere dovute ad un learning rate troppo elevato. Di conseguenza, si implementano i callbacks di EarlyStopping e ReduceLROnPlateau con le stesse caratteristiche utilizzate per il modello Dense.

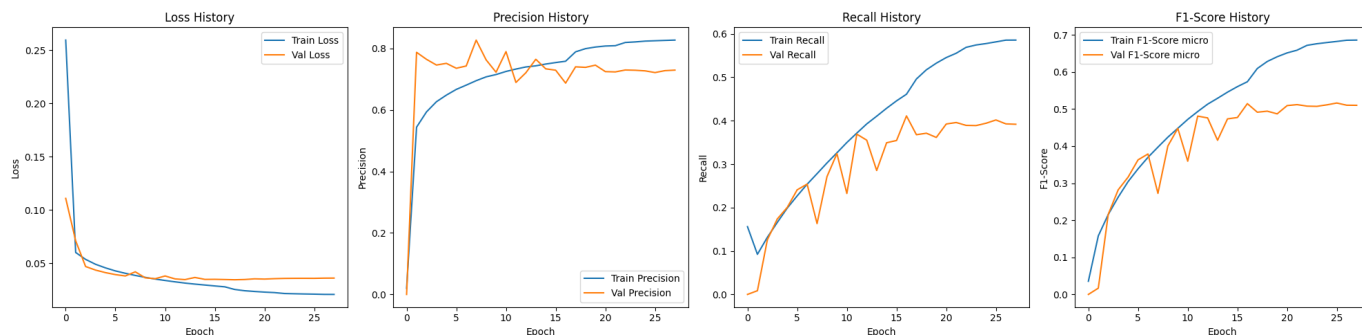


Figure 12: Curve di validazione e training di Loss, Precision, Recall e F1-Score(da sinistra a destra).

In figura 12 si osserva una riduzione progressiva delle oscillazioni man mano che il callback di ReduceLROnPlateau si attiva. L'Earlystopping ferma il training all'epoca 28, ripristinando i pesi dell'epoca 18. I valori delle metriche e di loss sui dati di validazione, che fanno riferimento alla 18° epoca sono:

Loss: 0.0346, Precision: 0.7401, Recall: 0,3677 e F1-Score: 0.4913.

Invece, sui dati di test si ottiene:

Loss: 0.0348, Precision: 0.7387, Recall: 0,3672 e F1-Score: 0.4906.

A questo punto si prova ad aumentare la capacità della rete. Si aumenta la dimensione del layer convoluzionale e se ne aggiunge un secondo identico al primo. Inoltre si aumenta la dimensione del penultimo layer Dense da 512 a 1024, sempre per evitare eventuali colli di bottiglia con l'ultimo layer. Infine si aumenta il vocabolario del layer TextVectorization da 10000 a 25000 e di conseguenza anche la dimensione di input del layer Embedding. Dopo questo layer viene anche inserito un layer di SpatialDropout1D. Infatti, aumentando la capacità della rete, aumenta il rischio di overfitting e viene mitigato con l'inserimento di questo layer. Lo SpatialDropout1D spegne interi canali di feature lungo tutta la sequenza, invece che spegnere singoli numeri come, ad esempio, farebbe un layer Dropout. Questo può essere utile perchè nelle sequenze di testo, i valori vicini all'interno di un vettore di embedding sono fortemente correlate. Quindi spengendo singoli numeri, le attivazioni vicine possono coprire quelle mancanti. Lo SpatialDropout costringe la rete a non fare affidamento su specifici canali semantici per riconoscere un pattern. Quindi, il modello sarà composto da:

- Embedding(input_dim=25001, output_dim=512)
- SpatialDropout1D(0.2)
- Conv1D(filters=256, kernel_size=5, activation='relu', padding='same')
- BatchNormalization()
- Conv1D(filters=256, kernel_size=5, activation='relu', padding='same')
- BatchNormalization(),
- GlobalMaxPooling1D()
- BatchNormalization(),
- Dense(1024, activation='relu')

- BatchNormalization(),
- Dropout(0.5)
- Dense(567, activation='sigmoid')

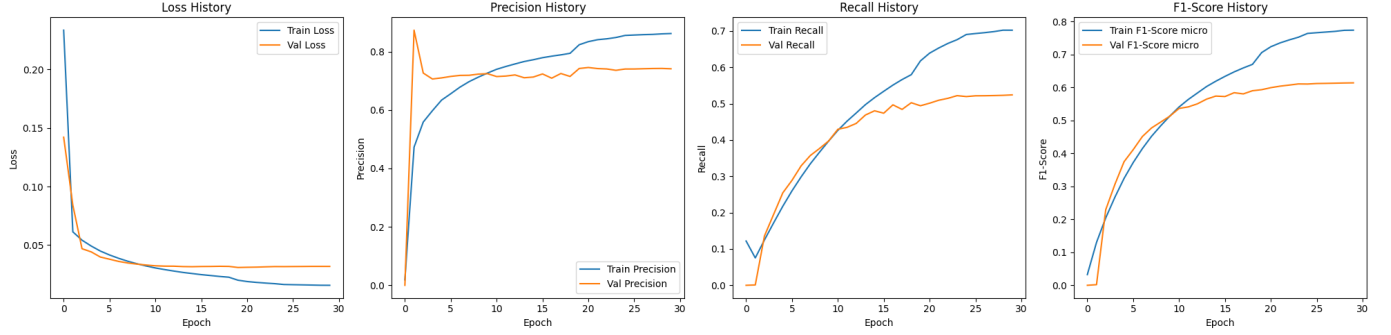


Figure 13: Curve di validazione e training di Loss, Precision, Recall e F1-Score(da sinistra a destra).

Dalla history del training di questo modello si ricavano curve di validazione molto più stabili rispetto a prima (fig:13). L'EarlyStopping ferma l'allenamento alla 30° epoca, reimpostando i pesi dell'epoca 20. Con i pesi così ripristinati si ottiene sui dati di validazione:

Loss: 0.0309, Precision: 0.7421, Recall: 0.4942 e F1-Score: 0.5933.

Sui dati di test si ottiene:

Loss: 0.0312, Precision: 0.7466, Recall: 0.4947 e F1-Score: 0.5951.

Rispetto ai valori ottenuti in precedenza, si ottiene un aumento significativo di Recall(da 0.3677 a 0.4942 sui dati di validazione). Come fatto per il modello Dense, si prova sostituire la BinaryCrossentropy con la Binary Focal Crossentropy. Anche in questo caso si valuta il modello variando i valori di γ e α .

γ	α	Loss	Precision	Recall	F1-Score
2.0	0.25	0,0089	0.7367	0.4689	0.5731
3.0	0.45	0.0048	0.7263	0.4413	0.5490
4.0	0.65	0.0028	0.7205	0.4210	0.5314
5.0	0.90	0.0015	0.6977	0.3996	0.5082

Table 5: Valori di Loss, Precision, Recall e F1-Score al variare dei parametri γ e α .

Ancora una volta si ottengono valori delle metriche minori rispetto all'utilizzo della semplice Binay-Crossentropy. Come fatto in precedenza si fissa il valore di γ a 2.0 e si varia il valore del parametro α , i valori delle metriche sono riportati in tabella 6.

α	Loss	Precision	Recall	F1-Score
0.25	0.0089	0.7367	0.4689	0.5731
0.50	0.0089	0.7329	0.4721	0.5743
0.75	0.0089	0.7356	0.4793	0.5805
0.90	0.0088	0.7398	0.4742	0.5779

Table 6: Valori di Loss, Precision, Recall e F1-Score al variare del parametro α fissato γ .

Si decide quindi di tornare ad utilizzare la BinaryCrossentropy come funzione di loss. Con questo modello si ottengono le predizioni sui dati di test e se ne valuta le prestazioni osservando i valori delle metriche al variare della soglia decisionale(tab:8 e fig:14), oltre alla media di keywords assegnate agli articoli(tab:7).

Soglia	0.30	0.35	0.40	0.45	0.50	0.55	0.60
Media	7.00	6.44	5.95	5.53	5.12	4.77	4.42

Table 7: Valori della media di keywords assegnate agli articoli dei dati di test.

Soglia	Precision	Recall	F1-Score
0.30	0.6524	0.5905	0.6199
0.35	0.6792	0.5660	0.6174
0.40	0.7042	0.5423	0.6127
0.45	0.7257	0.5187	0.6050
0.50	0.7466	0.4947	0.5951
0.55	0.7641	0.4714	0.5831
0.60	0.7813	0.4472	0.5688

Table 8: Tabella dei valori delle metriche al variare della soglia decisionale.

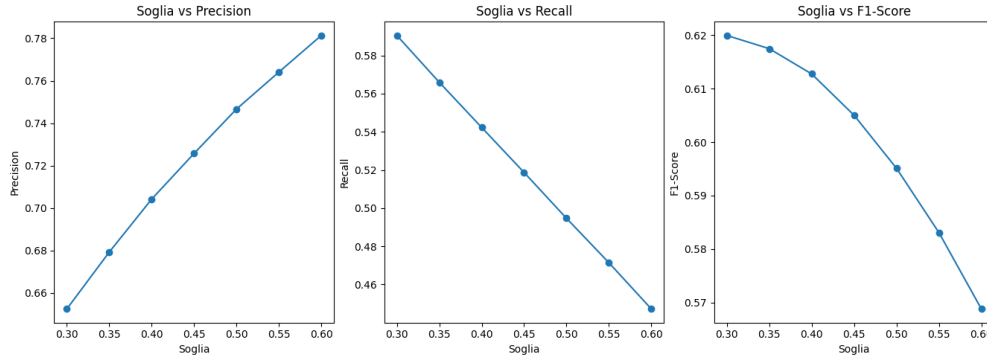


Figure 14: Grafici dei valori delle metriche Precision, Recall e F1-Score al variare della soglia decisionale.

Anche in questo caso si osserva un valore migliore di F1-Score per una soglia di 0.3. Con questa soglia si ottiene anche una media di keywords per articolo molto vicina a quella del dataset che è di 7.73. Di seguito (fig:15, 16, 17) si riportano alcuni esempi di predizioni fatte con questo modello.

```
Keywords predette con soglia 0.5: ('diffraction', 'elastic scattering', 'holography', 'pomeron', 'total cross section', 'unitarity')
Keywords predette con soglia 0.3: ('diffraction', 'elastic scattering', 'blueball', 'holography', 'pomeron', 'regge poles', 'total cross section', 'unitarity')
Keywords originali : ['total cross section', 'blueball', 'elastic scattering', 'pomeron', 'holography', 'diffraction', 'unitarity']
```

Figure 15: Predizione del modello CNN alla soglia di allenamento(0.5) e ottimale(0.3).

```
Keywords predette con soglia 0.5: ('boundary condition', 'cosmological model', 'curvature', 'inflation', 'inflaton', 'numerical calculations', 'potential scalar')
Keywords predette con soglia 0.3: ('boundary condition', 'cosmological model', 'curvature', 'inflation', 'inflaton', 'numerical calculations', 'potential scalar')
Keywords originali : ['curvature', 'quantum mechanics', 'potential scalar', 'inflaton', 'cosmological model', 'inflation', 'reheating', 'boundary condition']
```

Figure 16: Predizione del modello CNN alla soglia di allenamento(0.5) e ottimale(0.3).

```
Keywords predette con soglia 0.5: ('dark matter halo', 'density', 'star', 'tension', 'velocity')
Keywords predette con soglia 0.3: ('dark matter halo', 'density', 'galaxy', 'star', 'tension', 'velocity')
Keywords originali : ['velocity', 'star', 'tension', 'density', 'infrared', 'dark matter halo']
```

Figure 17: Predizione del modello CNN alla soglia di allenamento(0.5) e ottimale(0.3).

4 Modello LSTM

L'ultimo modello implementato è un modello composto da una parte formata da layer di tipo LSTM e una parte Dense finale. Come nei modelli precedenti il primo layer è di tipo Embedding e i testi vengono prima tokenizzati da un layer TextVectorization. Il modello, implementato in modalità Sequential, è quindi composto da:

- Embedding(input_dim=10001, output_dim=512)
- LSTM(64, return_sequences=False)
- Dense(512, activation='relu')
- Dense(567, activation='sigmoid')

La funzione di loss utilizzata è la BinaryCrossentropy, come optimizer si sceglie adam e si valutano le solite metriche: Precision, Recall e F1-Score. Si imposta un batch size di 256 e un numero di epoche pari a 50.

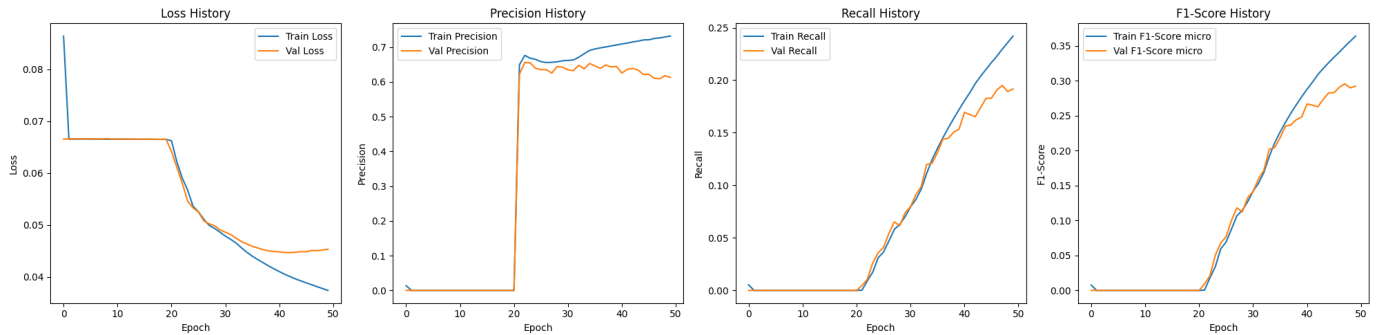


Figure 18: Curve di validazione e training di Loss, Precision, Recall e F1-Score(da sinistra a destra).

Come si osserva dalle curve di validazione e training (fig:18), per le prime 20 epoche la rete fatica a trovare i pattern necessari alla classificazione. Questo è dovuto all'aver impostato `return_sequences = False`. Con questa impostazione il layer fornisce in output un vettore che riassume il significato del testo processato. Se le informazioni necessarie si trovano all'inizio del testo, il loro segnale verrà attenuato dalle informazioni successive e la rete faticherà a trovare quel segnale. Di conseguenza, si imposta `return_sequences=True`; in questo modo il layer LSTM ritorna in output un vettore per ogni parola del testo. Questi vettori sono influenzati dal contesto nel quale sono inserite le parole, in particolare i vettori sono creati ricordando le parole precedenti. I layer Dense però non sono in grado di prendere in input tensori con queste dimensioni, allora si inserisce un layer di GlobalMaxPooling1D tra il layer LSTM e il primo layer Dense. Il MaxPooling sarà in grado di filtrare il tensore in uscita dal layer LSTM e selezionare solo i segnali più forti.

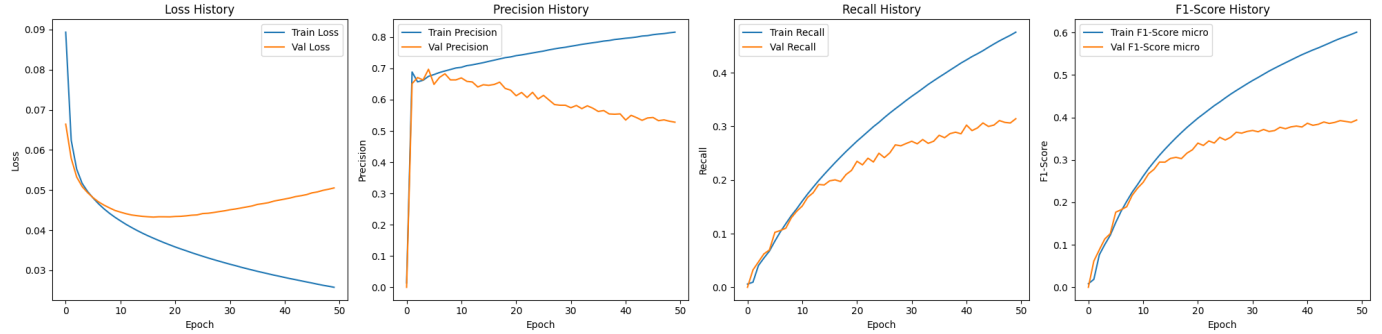


Figure 19: Curve di validazione e training di Loss, Precision, Recall e F1-Score(da sinistra a destra).

Si osserva (fig:19) come con questa configurazione la rete riesca ad apprendere i pattern fin dalle prime epoche. Si nota però che la rete va in overfitting dopo la decima epoca. Per contrastare l'overfitting si inseriscono layer di normalizzazione e di Dropout. Dopo il layer LSTM viene inserito un layer di tipo LayerNormalization, mentre dopo il layer di MaxPooling e l'hidden layer Dense vengono inseriti layer Batch-Normalization. Infine viene inserito un layer di Dropout tra i due layer Dense. A differenza del layer di BatchNormalization, il LayerNormalization è indipendente dalla dimensione del batch fornito alla rete. Con questo layer la normalizzazione viene eseguita su ogni articolo calcolando la media e la deviazione standard con le sole feature di quell'articolo. Allenando il modello così modificato, si ricava sui dati di validazione valori delle metriche pari a:

Loss: 0.0398, Precision: 0.7021, Recall: 0.3224 e F1-Score: 0.4419

Invece, sui dati di test si ottiene:

Loss: 0.0403, Precision: 0.7038, Recall: 0.3232 e F1-Score: 0.4430

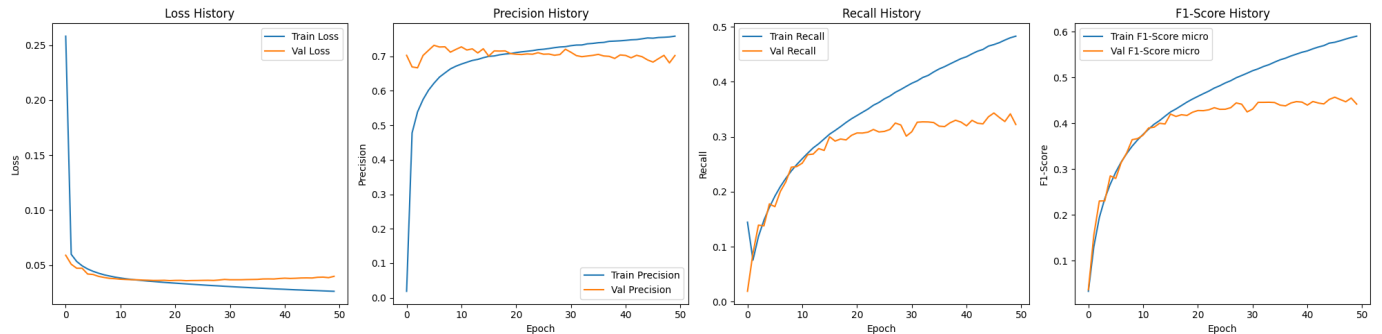


Figure 20: Curve di validazione e training di Loss, Precision, Recall e F1-Score(da sinistra a destra).

Dai grafici (fig:20) si riscontra una riduzione del gap tra la curva di validazione e quella di training, effetto dovuto ai layer di normalizzazione e di Dropout inseriti. Anche per questo modello si implementano i callbacks di EarlyStopping e ReduceLROnPlateau, come fatto con i modelli precedenti. Si aumenta il numero di epoche impostate a 100, questo perchè comunque il callback di EarlyStopping interromperà l'allenamento ripristinando i pesi dell'epoca in cui il valore di loss è migliore.

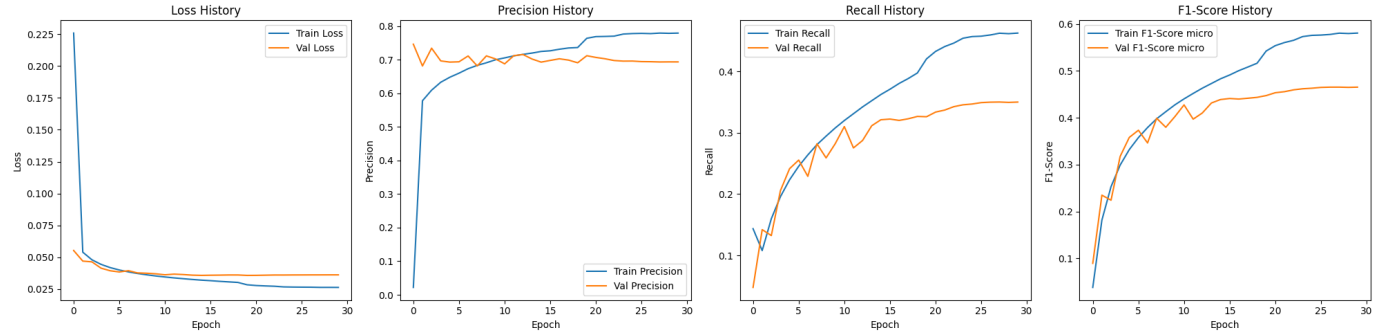


Figure 21: Curve di validazione e training di Loss, Precision, Recall e F1-Score(da sinistra a destra).

Nonostante le 100 epoche impostate l'EarlyStopping interrompe il training alla 30° epoca, ripristinando i pesi dell'epoca 20. Si osserva dai grafici (fig: 21) l'efficacia del ReduceLROnPlateau, infatti le oscillazioni che si osservano nelle prime epoche si riducono con la riduzione del learning rate. Con l'implementazione dei callbacks non si osservano cambiamenti significativi nei valori delle metriche, ma si ottiene valori della funzione di loss minori. Sui dati di validazione si ottiene:

Loss: 0.0358, Precision: 0.7119, Recall: 0.3263 e F1-Score: 0.4475.

Mentre sui dati di test si ricava:

Loss: 0.0363, Precision: 0.7094, Recall: 0.3256 e F1-Score: 0.4464.

Si prova, quindi, ad aumentare la capacità della rete. Per prima cosa si aumenta il vocabolario del TextVectorization da 10000 a 25000, di conseguenza si aumenta la dimensione di input del layer Embedding. Successivamente, si inserisce un secondo layer LSTM aumentandone la dimensione da 64 a 128 e rendendoli bidirezionali. In questo modo i layer LSTM saranno in grado di leggere il testo in entrambe le direzioni e non solo da sinistra verso destra. Inoltre, si rimuove il Dropout tra i layer Dense, inserendolo direttamente nei layer LSTM. Con questo Dropout verranno spenti i segnali di alcune parole del testo e la rete dovrà cercare di riconoscere i pattern senza fare affidamento su parole che possono essere fondamentali. Infine si modifica la dimensione dell'ultimo layer Dense per evitare colli di bottiglia. Quindi, la nuova struttura del modello sarà composta da:

- Embedding(input_dim=25001, output_dim=512)
- Bidirectional(LSTM(128, return_sequences=True, dropout=0.2))
- LayerNormalization()
- Bidirectional(LSTM(128, return_sequences=True, dropout=0.2))
- LayerNormalization()
- GlobalMaxPool1D()
- BatchNormalization()
- Dense(1024, activation='relu')
- BatchNormalization()
- Dense(567, activation='sigmoid')

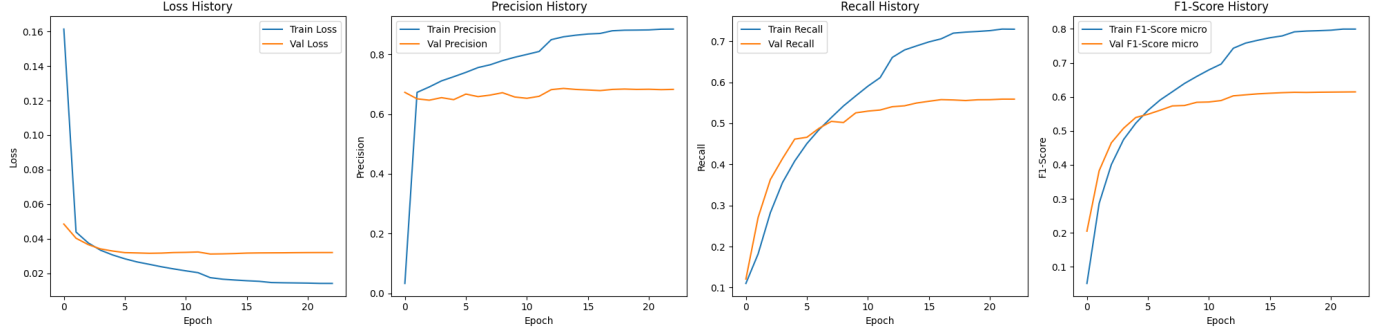


Figure 22: Curve di validazione e training di Loss, Precision, Recall e F1-Score(da sinistra a destra).

Dalle curve in figura 22 si osserva un aumento del gap tra validazione e loss, dovuto all'aumento dei parametri della rete. In questo caso l'EarlyStopping interrompe l'allenamento all'epoca 23, ripristinando i pesi dell'epoca 13. Con questo modello si ottengono valori migliori di Recall, anche se si osserva un leggero calo di Precision. Sui dati di validazione si ottiene:

Loss: 0.0311, Precision: 0.6818, Recall: 0.5405 e F1-Score: 0.6030.

Sui dati di test si ricava:

Loss: 0.0314, Precision: 0.6862, Recall: 0.5420 e F1-Score: 0.6057.

Come per i modelli precedenti si valuta l'utilizzo della Binary Focal Crossentropy variando i valori di γ e α . In tabella 9 vengono riportati i valori delle metriche ottenuti sui dati di validazione.

γ	α	Loss	Precision	Recall	F1-Score
2.0	0.25	0.0092	0.6581	0.5248	0.5839
3.0	0.45	0.0051	0.6440	0.5091	0.5687
4.0	0.65	0.0028	0.6309	0.4722	0.5402
5.0	0.90	0.0015	0.6183	0.4787	0.5396

Table 9: Valori di Loss, Precision, Recall e F1-Score al variare dei parametri γ e α .

Ancora una volta si osserva che con la Binary Focal Crossentropy si ottengono valori delle metriche peggiori. A questo punto si prova a lasciare fisso il parametro γ a 2.0 e si varia il parametro α .

α	Loss	Precision	Recall	F1-Score
0.25	0.0092	0.6581	0.5248	0.5839
0.50	0.0091	0.6556	0.5175	0.5784
0.75	0.0091	0.6617	0.5249	0.5854
0.90	0.0091	0.6632	0.5131	0.5786

Table 10: Valori di Loss, Precision, Recall e F1-Score al variare del parametro α fissato γ .

Anche fissando il parametro γ e variando α a valori più alti non si riscontrano miglioramenti significativi nei valori delle metriche, si sceglie dunque di utilizzare la BinaryCrossentropy. Come fatto in precedenza, si valuta il modello sui dati di test modificando i valori di soglia decisionale delle metriche utilizzate.

Soglia	0.30	0.35	0.40	0.45	0.50	0.55	0.60
Media	8.36	7.66	7.10	6.58	6.11	5.66	5.25

Table 11: Valori della media di keywords assegnate agli articoli dei dati di test.

Soglia	Precision	Recall	F1-Score
0.30	0.5881	0.6358	0.6111
0.35	0.6166	0.6111	0.6138
0.40	0.6401	0.5880	0.6129
0.45	0.6638	0.5651	0.6105
0.50	0.6862	0.5420	0.6057
0.55	0.7071	0.5179	0.5979
0.60	0.7261	0.4933	0.5875

Table 12: Tabella dei valori delle metriche al variare della soglia decisionale.

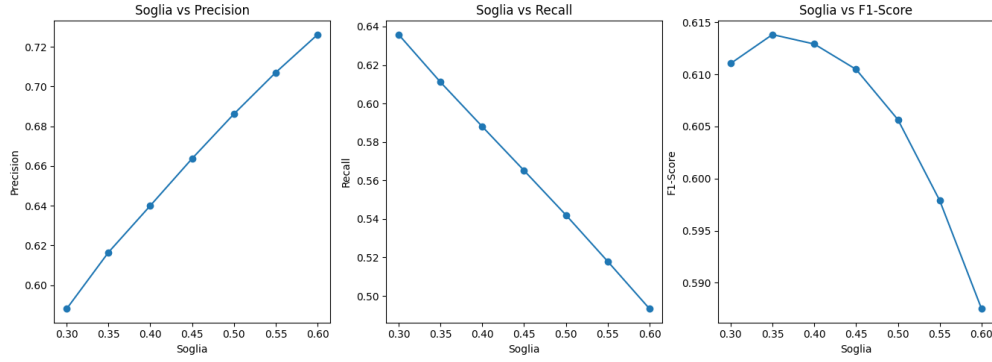


Figure 23: Grafici dei valori delle metriche Precision, Recall e F1-Score al variare della soglia decisionale.

Dalle tabelle 11 e 12 si ha che la soglia ottimale è 0.35. Di seguito (fig:24, 25 e 26) si riportano alcuni esempi di predizioni fatte dalla rete sia con soglia pari a 0.5 che con la soglia ottimale di 0.35; che vengono confrontate con quelle originali.

```
Keywords predette con soglia 0.50: ('quantum chromodynamics', 'renormalization group')
Keywords predette con soglia 0.35: ('momentum transfer', 'quantum chromodynamics', 'quark mass', 'renormalization group')
Keywords originali : ['quantum chromodynamics', 'renormalization group', 'perturbation theory', 'momentum transfer', 'bibliography', 'quark mass']
```

Figure 24: Predizione del modello Dense alla soglia di allenamento(0.5) e ottimale(0.35).

```
Keywords predette con soglia 0.50: ('cosmic background radiation', 'gamma ray', 'glast', 'icecube', 'neutrino flux', 'tension')
Keywords predette con soglia 0.35: ('annihilation', 'cosmic background radiation', 'gamma ray', 'glast', 'icecube', 'neutrino flux', 'photon', 'tension')
Keywords originali : ['gamma ray', 'neutrino flux', 'annihilation', 'icecube', 'cosmic background radiation', 'tension', 'two photon']
```

Figure 25: Predizione del modello LSTM alla soglia di allenamento(0.5) e ottimale(0.35).

```
Keywords predette con soglia 0.50: ('magnetic field external field', 'neutrino magnetic moment', 'symmetry chiral')
Keywords predette con soglia 0.35: ('hamiltonian', 'magnetic field external field', 'neutrino magnetic moment', 'neutrino oscillation', 'symmetry chiral')
Keywords originali : ['magnetic field external field', 'symmetry chiral', 'neutrino magnetic moment', 'dispersion relation', 'hamiltonian']
```

Figure 26: Predizione del modello LSTM alla soglia di allenamento(0.5) e ottimale(0.35).

5 F1-Score

Come già detto nelle sezioni precedenti, l'F1-Score è calcolato come la media armonica tra Precision e Recall del modello. Si può quindi provare a monitorare questa metrica nei callbacks, cambiando la modalità da quella di default a max. Si cercano infatti i pesi dell'epoca alla quale si ottiene il valore maggiore di F1-Score. Nelle tabelle 13, 14, 15 sono riportati i valori delle metriche per i tre modelli e nelle figure 27, 28, 29 le curve di validazione e training ottenute dalla history del modello.

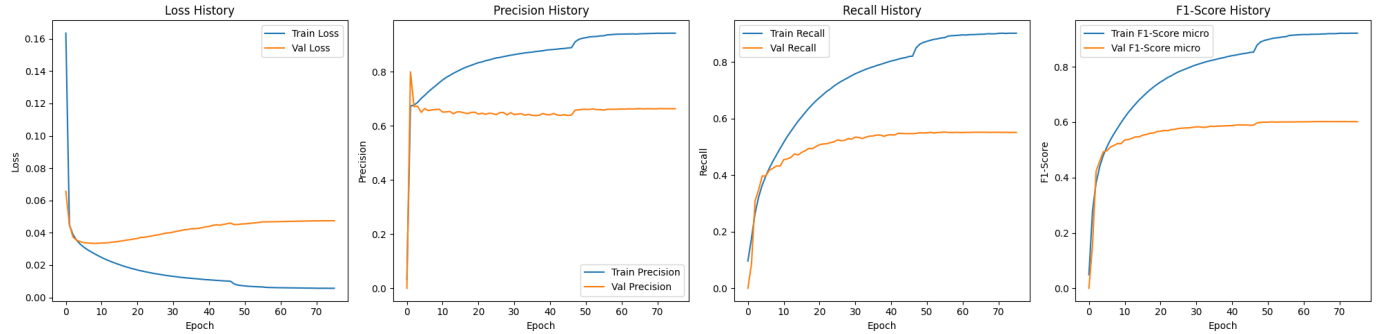


Figure 27: Curve di validazione e training di Loss, Precision, Recall e F1-Score(da sinistra a destra), ottenute monitorando l'F1-Score del modello Dense.

Monitor	Loss	Precision	Recall	F1-Score
Loss	0.0327	0.6724	0.4739	0.5560
F1-Score	0.0472	0.6634	0.5514	0.6022

Table 13: Valori delle metriche e della loss ottenuti monitorando la loss e l'F1-Score del modello Dense.

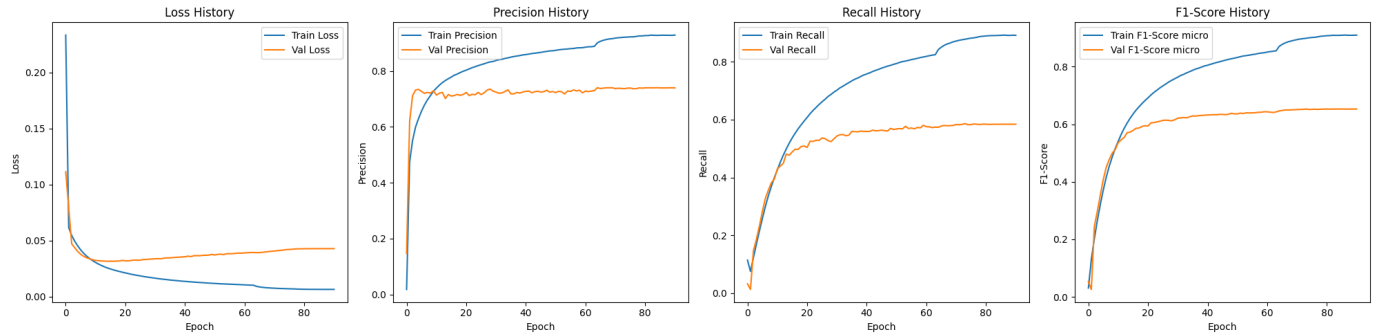


Figure 28: Curve di validazione e training di Loss, Precision, Recall e F1-Score(da sinistra a destra), ottenute monitorando l'F1-Score del modello CNN.

Monitor	Loss	Precision	Recall	F1-Score
Loss	0.0309	0.7421	0.4942	0.5933
F1-Score	0.0427	0.7397	0.5844	0.6529

Table 14: Valori delle metriche e della loss ottenuti monitorando la loss e l'F1-Score del modello CNN.

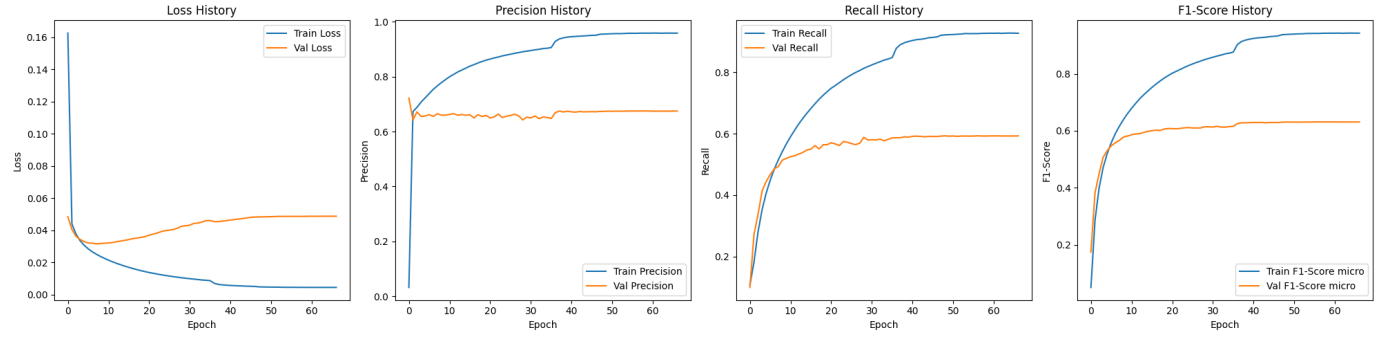


Figure 29: Curve di validazione e training di Loss, Precision, Recall e F1-Score(da sinistra a destra), ottenute monitorando l’F1-Score del modello LSTM.

Monitor	Loss	Precision	Recall	F1-Score
Loss	0.0311	0.6818	0.5405	0.6030
F1-Score	0.0486	0.6745	0.5932	0.6313

Table 15: Valori delle metriche e della loss ottenuti monitorando la loss e l’F1-Score del modello LSTM.

Per tutti i modelli si nota un calo poco significativo di Precision a discapito di un aumento considerevole di Recall. Dalle curve di validazione e training si osserva però un forte overfitting, si decide quindi di continuare ad utilizzare i modelli allenati monitorando la BinaryCrossentropy.